

Software Task Round

Autonomous Ground Vehicle

For the selections, **six (6)** tasks are given. For each task, you first need to understand the problem statement, read concepts from the internet (some resources are provided in this doc itself), and then implement (code) it.

The task explanations are given at the end of this document in a separate section.

1. Make your programs Object Oriented if possible, although this is not a requirement.
2. You are expected to maintain proper documentation in notion for each task even if you are not able to complete them.
3. The resources given are not sufficient and you are expected to find more on your own.
4. You may discuss among yourselves and help each other, but sharing your code is a strict no. The use of plagiarized code will result in complete and immediate disqualification for both candidates, from whom the code has been copied and who have copied it.
5. For the tasks that require it, you can visualize the path or data using Matplotlib or OpenCV.
6. Participants may utilize any resources – including AI tools (LLMs), books, or videos – provided they fully document all sources and demonstrate understanding of their application.
7. Don't hesitate to contact any of us in case of any obstacles.
8. We have allotted the time with sufficient consideration for various factors and hence cannot give any extension to anyone as it would be extremely unfair to the rest of the candidates

A word of advice- You are expected to do at least one task, but you are encouraged to do more - this will increase your chances of getting selected and will also help you explore your interests in different fields we work on and get a general idea.

Mandatory Docker Setup

This is compulsory for everyone. Read up on basic docker documentation - It will help you in copying local files to the container and do other basic maneuvers. You must do all tasks in the docker container and push them finally for evaluation.

Windows Setup:

- Docker Installation: Install Docker Windows
- Install VcXsrv: VcXsrv is an X server that enables GUI applications from a Docker container to be displayed on Windows. Launch XLaunch and configure it with the default suggested options to start the X server
- Run the Docker Container with GUI Access:

```
docker run --name agvdocker -e DISPLAY=host.docker.internal:0 -it agviitkgp/task:24
```

macOS Setup:

- Docker Installation: Install Docker macOS
- Install XQuartz: Restart your system after installing XQuartz, (reference)
- Run the Docker Container with GUI Access:

```
docker run --name agvdocker -e DISPLAY=$(ipconfig getifaddr en0):0 -it agviitkgp/task:24
```

Running Docker:

- After the first run, use this command to restart and attach to the container:

```
docker start -ai agvdocker
```

- Exit the Docker Safely:

```
exit
```

- VSCode Setup: Refer this link

1. Aw Shii, Here we flow again...

Seeing Motion Like Humans Do

Think of merging onto a highway at dusk: your brain instinctively tracks the speed of approaching cars using subtle visual cues, not raw speedometers. Optical flow replicates this biological motion sense for machines, solving critical problems other sensors miss.

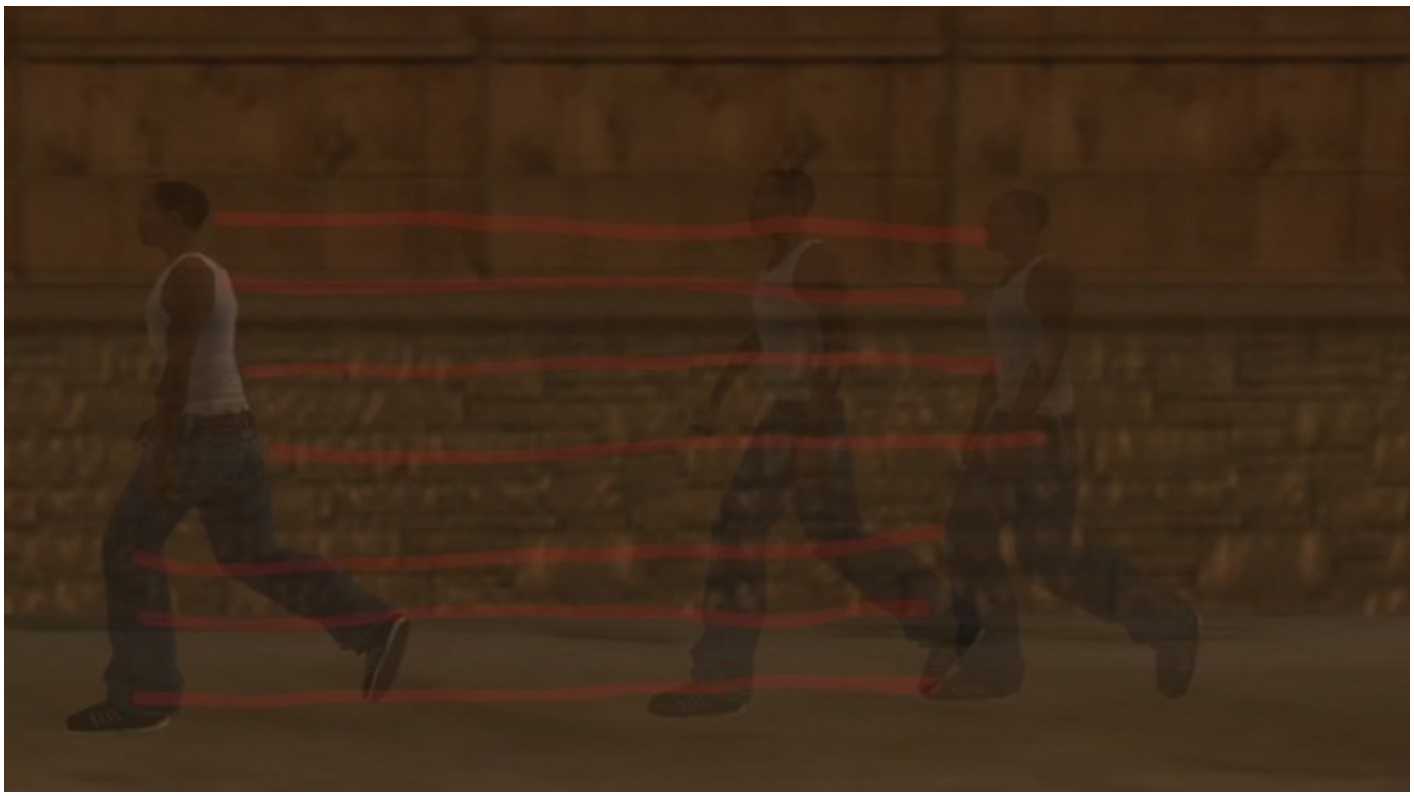
- Sudden Obstacles: Like spotting a skateboarder cutting across traffic (as in Chalmers University's collision detection system), optical flow detects approaching objects 30% faster than frame-by-frame detection.
- Parking Lot Navigation: Ever parallel park by gauging car bumpers' movement in your mirrors? Flow vectors create real-time "distance maps" between vehicles.
- Rainy Night Driving: New methods like SemARFlow maintain 85% accuracy in downpours where LiDAR fails—critical when windshield wipers struggle.
- Key Insight: Humans use motion cues first, details later—optical flow brings this biological advantage to machine.



SUBTASK 1

CJ NEEDS HELP!: IMPLEMENTING OPTICAL FLOW

You are expected to implement the Lucas-Kanade sparse optical flow estimation algorithm specified in this article on clips of your choice from this video. You are allowed to use basic OpenCV functions: *cv2.imread*, *cv2.imshow*, etc., and you may additionally use *cv2.goodFeaturesToTrack*. You may refer OpenCV documentation.



BONUS: Implement dense Optical Flow estimation algorithm on the video.

SUBTASK 2

GOING WITH THE FLOW

Let us now dive into a very interesting application of optical flow methods: moving your ego (a robot or an autonomous car) towards a target while avoiding obstacles. Your task here is to use the Lucas-Kanade algorithm in conjunction with a few methods in order to implement a make-shift controller that reaches a given goal in a cool simulator.

Before proceeding with the task, you are required to pull and set up the docker container as mentioned in the compulsory task. Operate only from the home directory.

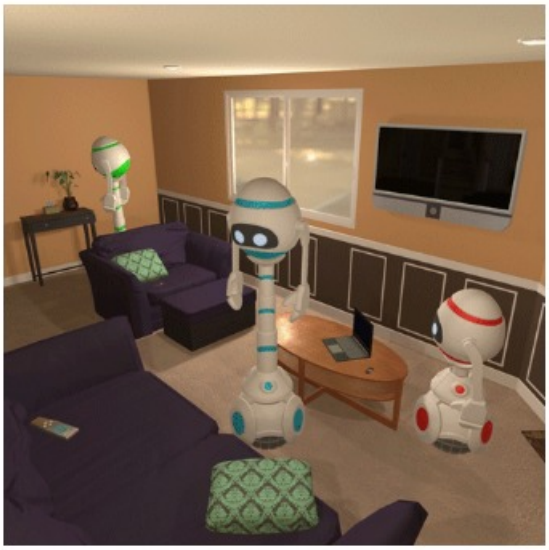
Launch the container and do a pip install of ai2thor, then create a python file where you will be coding up most or all of the following task.

Objective: Use sparse optical flow (Lucas-Kanade method) to navigate to any target coordinate in the configuration space in an AI2-THOR indoor environment (e.g., kitchens, living rooms).

You must follow these steps (refer to the paper linked below for more clarity):

- Capture RGB frames from AI2-THOR's agent camera.
- Compute optical flow between consecutive frames to identify motion vectors.
- Detect Focus of Expansion (FOE) to infer agent motion direction and distinguish ego-motion from obstacle motion.
- Generate a visual potential field where obstacles create repulsive vectors and the target (e.g., a doorway) creates an attractive vector.
- Integrate with AI2-THOR's API to steer the agent using gradient-based control.

You must go through this paper and understand its math (at least at a high-level) to accomplish this. Apart from your implementation of Lucas-Kanade method, you may use any library function from OpenCV and NumPy. Go through their documentation - use as many resources as you want (*including LLMs*) for this sub-task.





2. I Won't Hit On You !

Introduction

In the realm of autonomous vehicles, the ability to navigate safely through dynamic environments is crucial for reliable and efficient operation. One key component of this capability is dynamic obstacle avoidance, which involves detecting, tracking, and maneuvering around moving objects in real-time. This process plays a vital role in enabling autonomous vehicles to make split-second decisions, prevent collisions, and ensure the safety of both passengers and other road users.

Running the Simulator
Follow the instructions provided in the mandatory task to set up Docker and Visual Studio Code for the simulation environment. After making necessary modifications to the controller file, execute the following command to run the simulator:

./simulation.sh

The tasks in this simulation involve implementing and fine-tuning controllers for an autonomous system navigating through a simulated environment. The objective is to develop a controller capable of handling static waypoints, dynamic obstacles, and noisy sensor data. Below are the three tasks:

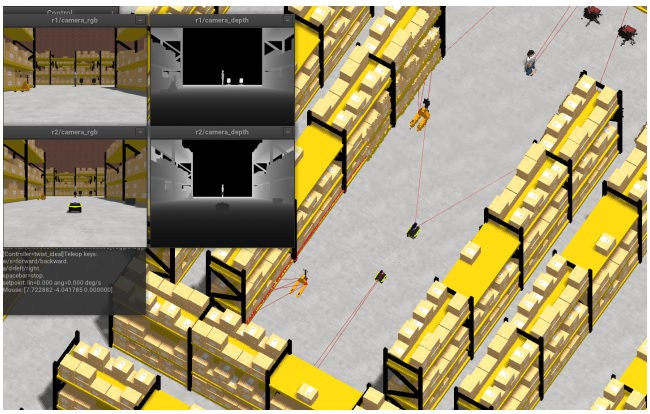
Task 1: Basic Static Controller Implementation

- Set the static input parameter to **true**. Implement a basic static controller, such as Follow the Gap, PID, or Pure Pursuit, to follow waypoints in this file:

  /mvsim_ros_noetic/catkin_ws/static_controller.py

- All required data is pre-subscribed and available within the file; refer to the provided comments for guidance. Tune the controller parameters to ensure smooth navigation.





Task 2: Dynamic Obstacle Avoidance Controller

- Set the static input parameter to **false** and the noise input parameter to **false**. Implement **a dynamic obstacle avoidance controller** utilizing LiDAR input, in this file:

  /mvsim_ros_noetic/catkin_ws/dynamic_controller.py

- The simulation environment will contain both dynamic and static obstacles. Select an appropriate algorithm to optimize performance and robustness against edge cases. Be as creative with this as you can.

Task 3: Noisy LiDAR-Based Dynamic Obstacle Avoidance

- Set the static input parameter to **false** and the noise input parameter to **true**. Modify the dynamic obstacle avoidance controller to function effectively in an environment with **noisy LiDAR data**. Implement the controller in the file:

  /mvsim_ros_noetic/catkin_ws/dynamic_controller.py

- Adapt the controller to handle real-world sensor noise while maintaining accurate navigation and obstacle avoidance.

3. Lost & Found: Multi-Agent Rendezvous

When Worlds Collide (Literally)

Imagine two search robots deployed in a disaster zone with partial maps - their survival depends on merging perspectives like human rescuers comparing mental maps. This task captures the algorithmic core of collaborative autonomy, solving critical coordination problems single agents can't.

- Urban Search & Rescue: Like firefighters converging through collapsed buildings (per DARPA SubT challenges), map merging enables rendezvous without GPS 37% faster than beacon systems.
- Swarm Construction: ETH Zurich's flying builders achieve 92% accuracy in shared map alignment for collaborative assembly.
- Key Insight: Nature solves this through ant pheromone trails and wolf pack triangulation - digital agents must achieve similar spatial consensus through algorithmic choreography.

Task: Build An Autonomous Multi-Agent Navigation Pipline

You are given a **custom-made simulator** in PyGame that works with the following 6 files - *pipeline_task.py*, *robot_api.py*, *robot_controller.py*, *map_merger.py*, *localization.py* and *planning.py*. The simulator can be run using the *robot_controller.py* file.

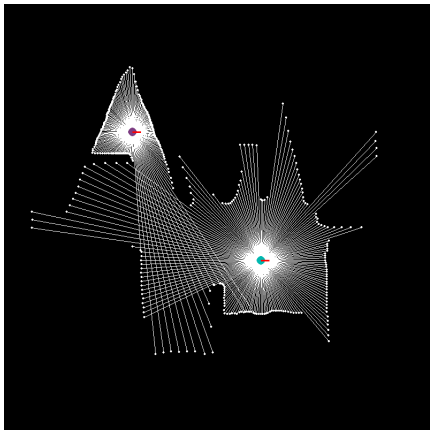
The task is designed broadly for you to understand the concepts behind **localization, map-merging and autonomous navigation or planning** in GPS-denied environments. The two robots in the environment of the simulator are equipped with sensors like LiDAR and IMU and the movement of the agents are controlled by a simple controller (you need not implement your own controller) for which the *robot_api.py* file provides API calls that can be used.

You may refer to the *pipeline_task.py* file which has five inter-related and progressively difficult subtasks that you may proceed to implement within the same file. The instructions for doing the tasks are also given in the same file.

Templates for the Planner, MapMerger, and Localization classes are provided in the

drive link. Your task is to complete the implementation for each class—implementing the planner, map merger, and localization functionalities respectively. You may modify the code within these classes as needed. Finally, integrate and use these classes within the Pipeline class.





Deliverables (What we evaluate for):

- *How well-organized your pipeline is.* A major problem in non-robust solutions arise when one part of the pipeline or code fails - either due to a bug in the code itself or due to extrinsic factors - like compute limitations or other real-life deployment failures. Your code must be built to handle such scenarios where one failure doesn't impact the other parts.
- *Understanding and Originality.* Your solution must be well-thought-out and researched. And by virtue of the same, you must have complete understanding of your pipeline as well as implementation details of your code.
- *Generalizability and Corner Case Testing.* We will stress test your solution. So, lazily put-together solutions built solely through clever (or even worse unclever) prompt engineering may not pass these cases. On other occasions, stress testing may also reveal bugs in say, code that you have written by yourself. The real reason for this deliverable is thus not only for weeding out lazy solutions, but also to test if you have accounted for corner cases that may very well happen during real-life deployment.
- *Novelty.* Introducing very rarely used algorithms that turn out to be efficient or coding up some new changes to existing popular algorithms will surprise us. So, be creative and innovative!

4. VizDoom

Introduction

While the task that follows below does not directly relate to autonomous vehicles, a lot of the ideas and concepts that you will use to solve it will directly carry over to autonomous vehicles and automation in general. This includes but is not limited to planning in continuous spaces, designing controllers to carry out your planned trajectory, and using simulators and integration so that you can test out your algorithms before deploying them in the real world.

For this task, you will be using ViZDoom, which is an AI research platform/simulator based on the game which arguably birthed the FPS genre - Doom. You can visit their GitHub repo here. Follow the instructions there to set it up on your machine (which is just a simple pip install for the most part) and go through the documentation and examples on how to use it. The overall goal of this task is to navigate a maze and reach a checkpoint.

Level 1

Load this custom .wad file into the simulator. A WAD file is a game data file used by Doom and Doom II, as well as other first-person shooter games that use the original Doom engine. On correctly loading the .wad file, you will see something like this:



Global Planning

You can either use the automap from the simulator or use the map directly from here. The white pixel is the initial position of your ego and blue is where you will find the blue skull needed to finish the level. You can use any planning algorithm you like to plan a trajectory but we prefer RRT*. You will need to keep an eye out for dynamic and kinetic constraints of your ego since any path connecting the two points might not be feasible for your controller to follow in the next level.

Trajectory Following

You will need to now translate your global trajectory to actions in space and direct your ego to the blue skull. You might need a closed loop controller that corrects the errors that can build up in following the trajectory.

Level 2

You will no longer have access to the automap or a predefined map. You are only allowed to use the data from the buffers. The objective remains the same. You will most likely need to implement some sort of DFS search in the space while keeping approximate track of where you are, controls for backtracking and behaviors to search for open passages; however, you are free to take any approach you like. An example depth buffer is given below:



Submission

To make your submission for this task, use this modified version of the ViZDoom repository. The examples folder contains two python files, for Level 1 and Level 2 of this task. You can add your code in these files and submit them. We have already set up the .wad file of the custom map in this repository and made changes in the configuration files accordingly.

Feel free to run and change the existing scripts in the examples folder to get a better understanding of the environment.

In addition to the mentioned codes, submit an image of the final global path that your designed algorithm suggested for the given map image. Submit a screen recording of the next parts of the task.



5. Locate Me If You Can

When Millimeters Matter

Imagine an autonomous hospital delivery robot navigating ICU corridors - one flawed sensor reading could mean collision with critical equipment. This challenge captures the essence of robust localization: transforming noisy LiDAR data into millimeter-precision positioning without GPS, just as surgeons rely on steady hands over shaky visuals.

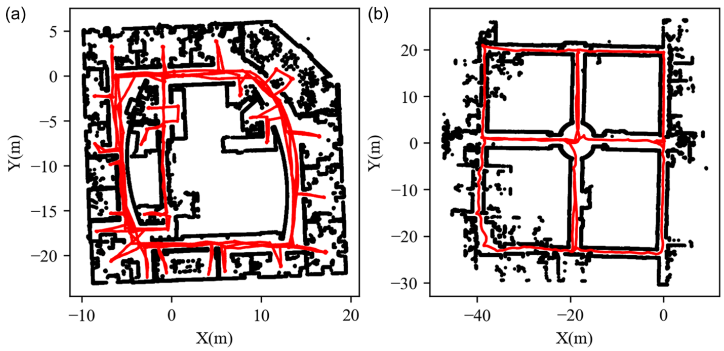
Task: Correlative Scan Matching for Localization

The task involves reading and implementing the research paper titled "A 2D-LiDAR-based localization method for indoor mobile robots using correlative scan matching" by Song Du, Tao Chen, Zhonghui Lou, and Yijie Wu (link to the research paper). You will have to implement the localization method using the correlative scan matching (CSM) technique on this dataset.

SUBTASK 1: Use the dataset above with the noise parameters of your choice to localize the agent on this map (use OpenCV to replace green pixels with white ones). Here are basic instructions on how to read the data.

SUBTASK 2: Utilize this solution to estimate better noise parameters and provide your rationale for doing so.

NOTE: Read the Research Paper thoroughly for the interview even if you are facing difficulties implementing the paper. If you don't understand exactly what a part of the paper describes, try to make it work by implementing something similar.



6. 3D Reconstruction

Introduction

In the realm of autonomous ground vehicles, the ability to perceive and understand the surrounding environment accurately is crucial for safe and efficient operation. One key component of perception is 3D reconstruction, which involves creating a detailed and comprehensive representation of the scene in three dimensions. This process plays a vital role in enabling autonomous vehicles to make informed decisions, accurately detect objects, navigate complex environments, and effectively plan their trajectories.

Task

This task requires you to construct 3D representation of surroundings from multiple view images. A detailed description of the concept behind the task and how you are expected to tackle it are given in the Link. All the required data is also given in the above link.



3D Reconstruction

Contact Information

Name	Phone Number	Email
Anjali Deepu	+91 94965 11112	mailanjalideepu@gmail.com
Arnav Singh	+91 97736 49266	arnavs986@gmail.com
Aryan Raina	+91 97797 75943	aryan.raina000@gmail.com
Damiya Gondha	+91 94089 75397	damiyagondha@gmail.com
Sandip Das	+91 77187 18130	sandipdas202@outlook.com
Theyanesh E R	+91 80724 71330	theyanesher@gmail.com